Copyright All rights reserved

16. 자료와 파일 입출력
(Data Structure and File I/O)

Heesung Oh
https://github.com/3dapi

16.1 스트림(Stream)

Heesung Oh
Copyright All rights reserved

- 스트림
  - 연속된 바이트의 흐름
  - 입출력 대상 → 스트림(Stream) 으로 표현
  - 장치 독립성
    - 파일, 콘솔, 네트워크 등 다양한 장치들의 입출력을 동일한 방식으로 처리
    - Ex) 파일 출력 , 콘솔 출력 → 코드 구조 유지

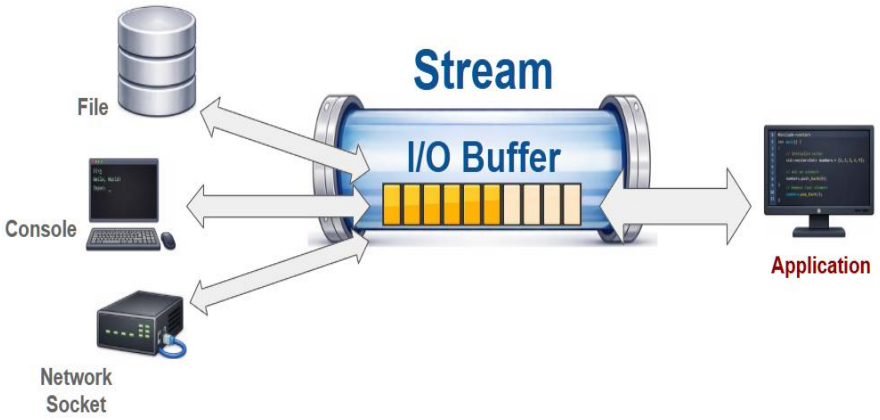
- 스트림과 버퍼
  - 스트림은 내부적으로 버퍼(Buffer: 임시 저장 공간) 사용
  - 버퍼를 사용하는 이유

    CPU/메모리 → 매우 빠름 , 디스크/네트워크 → 느림
    → 속도 차이 발생 → 버퍼로 데이터를 모아서 처리
    → 장치 접근 횟수 감소 → 성능 향상

16.1 스트림(Stream)

Heesung Oh
Copyright All rights reserved

- 출력 동작
  - 프로그램 → 데이터를 버퍼에 저장
  - 다음 경우 실제 장치로 전달됨
  - 버퍼가 가득 찼을 때
  - 개행 문자 출력 시
  - fflush 호출 시
  - 스트림 종료 시



- 입력 동작
  - 장치 → 데이터를 한 번에 읽어서 버퍼 저장
  - 프로그램 → 버퍼에서 필요한 만큼 사용
  - 장치 접근 횟수 감소

16.1 스트림(Stream)

Heesung Oh
Copyright All rights reserved

- **스트림 종류**
  - 텍스트 스트림
    - 문자 기반 입출력
    - 사람이 읽기 쉬움
  - 바이너리 스트림
    - 데이터를 그대로 입출력
    - 구조체, 게임 데이터 등에 사용

- **표준 스트림**
  - 프로그램 시작 시 자동 생성
  - stdin : 표준 입력
  - stdout : 표준 출력
  - stderr : 표준 에러 출력

- `int fflush(FILE *stream)`
  - 출력 버퍼를 즉시 비움

```
printf("Loading...");
fflush(stdout);    // 즉시 화면에 출력
```

16.2 파일 스트림 생성/종료 fopen, fclose

Heesung Oh
Copyright All rights reserved

- **파일 스트림의 생성과 종료**
  - 파일은 운영체제가 관리하는 시스템 자원
    → 사용 후 반드시 반환 필요 (fclose) → 장시간 실행 시 자원 누수 발생

- **자원 해제**
  - 동적 메모리: malloc → free
  - 파일 스트림: fopen → fclose

- **기본 흐름**
  - fopen (열기) → 읽기 / 쓰기 → fclose (닫기)

  - 내부 동작
    - OS: 파일 디스크립터, 버퍼 등 자원 할당
      → 파일 스트림(FILE) 으로 관리

16.2.1 파일 열기 모드

Heesung Oh
Copyright All rights reserved

- Windows 텍스트 모드 주의
  - 줄바꿈 자동 변환 (₩n ↔ ₩r₩n) → 바이너리 모드 사용

모드	읽기	쓰기	파일 존재 조건	텍스트/바이너리	파일에 저장된 기존 내용
"r"	**O**	**X**	**반드시 존재해야 함**	**텍스트 모드 (rt와 동일)**	**기존 데이터 유지**
"w"	**X**	**O**	**없으면 생성**	**텍스트 모드 (wt와 동일)**	**기존 데이터 모두 삭제(truncate) 후 새로 작성**
"a"	**X**	**O**	**없으면 생성**	**텍스트 모드 (at와 동일)**	**기존 데이터 유지 (파일 끝에 추가)**
"r+"	O	O	반드시 존재해야 함	텍스트 모드 (r+t와 동일)	기존 데이터 유지
"w+"	O	O	없으면 생성	텍스트 모드 (w+t와 동일)	기존 데이터 모두 삭제(truncate) 후 새로 작성
"a+"	O	O	없으면 생성	텍스트 모드 (a+t와 동일)	기존 데이터 유지 (쓰기 위치는 항상 끝)
"rb"	**O**	**X**	**반드시 존재해야 함**	**바이너리 모드 (b 지정)**	**기존 데이터 유지**
"wb"	**X**	**O**	**없으면 생성**	**바이너리 모드 (b 지정)**	**기존 데이터 모두 삭제(truncate) 후 새로 작성**
"ab"	X	O	없으면 생성	바이너리 모드 (b 지정)	기존 데이터 유지 (파일 끝에 추가)
"r+b"	O	O	반드시 존재해야 함	바이너리 모드 (b 지정)	기존 데이터 유지
"w+b"	O	O	없으면 생성	바이너리 모드 (b 지정)	기존 데이터 모두 삭제(truncate) 후 새로 작성
"a+b"	O	O	없으면 생성	바이너리 모드 (b 지정)	기존 데이터 유지 (쓰기 위치는 항상 끝)

16.2.1 파일 열기 모드

Heesung Oh
Copyright All rights reserved

〈파일 열기 모드 사용 예제〉

```
// 1. 파일 열기 (쓰기 모드)
FILE* fp = fopen("example.txt", "w");

// 2. 파일 유효 체크
if (!fp)
{
    printf("fopen 실패\n");
    // 또는
    // perror("fopen 실패");
    return error;
}

// 3. 파일에 데이터 기록
fprintf(fp, "Hello File\n");

// 4. 파일 닫기 (자원 반환)
fclose(fp);
fp = NULL;
```

16.2.2 EOF

Heesung Oh
Copyright All rights reserved

- End Of File
  - 더 이상 읽을 데이터가 없는 상태
  - 음수 값(보통 -1)으로 정의

```
#define EOF (-1)
```

- 입력 함수 (fgetc, getchar 등)
  - 반환형이 int → 데이터 없음 시 EOF 반환

- 콘솔 EOF 입력
  - Windows: Ctrl + Z → Enter
  - Linux/macOS: Ctrl + D

16.2.2 feof / ferror

Heesung Oh
Copyright All rights reserved

- `int feof(FILE* stream)`
  - 스트림이 파일 끝(EOF)에 도달한 경우 0이 아닌 값 반환

- `int ferror( FILE* stream)`
  - 스트림에서 읽기/쓰기 오류가 발생한 경우 0이 아닌 값 반환

```
while (1)
{
  bytes_read = fread(..., fp);
  if (bytes_read == 0)
  {
    if (feof(fp))
      printf("파일 끝(EOF)에 도달했습니다.\n");
    else if (ferror(fp))
      printf("읽기 오류가 발생했습니다.\n");

    break;
  }
```

16.3 형식 기반 입출력 함수 계열

Heesung Oh
Copyright All rights reserved

- 형식 기반 출력 함수

함수	대상	설명
printf	표준 출력 (stdout)	콘솔 화면에 형식에 맞춰 출력
fprintf	파일 스트림 (FILE*)	지정한 파일에 형식에 맞춰 출력
sprintf	메모리 버퍼 (char*)	문자열 버퍼에 형식에 맞춰 저장
snprintf	메모리 버퍼 (char*)	버퍼 크기를 제한하여 안전하게 저장

- 형식 기반 입력 함수

함수	대상	설명
scanf	표준 입력 (stdin)	콘솔 입력을 형식에 맞게 읽음
fscanf	파일 스트림 (FILE*)	파일에서 형식에 맞게 읽음
sscanf	메모리 문자열 (char*)	문자열을 형식에 맞게 해석(파싱)

16.3.1 형식 기반 입출력 예제

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

int main(void)
{
    int level = 5;
    double hp = 98.5;

    // printf
    printf("Level: %d, HP: %.1f\n", level, hp);

    FILE* fp = fopen("status.txt", "w");
    if (!fp)
        return 1;

    int gold = 100;

    // fprintf
    fprintf(fp, "Gold: %d\n", gold);
    fclose(fp);

    char buffer[128] = {0};
    int value = 123456789;

    // sprintf
    sprintf(buffer, "%d", value);

    printf("%s\n", buffer);
```

16.3.2 snprintf

Heesung Oh
Copyright All rights reserved

- `int snprintf (char* const buffer, size_t const count, char const* const format,...)`

- 동작
  - 주어진 크기 내에서 문자열 저장
  - 최대 `count-1` 문자 + `'\0'` 기록
  - 버퍼 범위를 넘지 않음
  - 문자열이 잘릴 수 있음

- 반환값
  - 실제로 출력하려고 했던 전체 문자열 길이 반환

```
#include <stdio.h>

int main(void)
{
  char buffer[5];
  int value = 123456789;

  int len = snprintf(buffer, sizeof(buffer)
                     , "%d", value);
  if (len >= sizeof(buffer))
  {
    printf("문자열이 잘렸습니다.\n");
  }

  printf("%s\n", buffer);
  return 0;
}
```

16.3.3 형식 기반 입력 – scanf 계열

Heesung Oh
Copyright All rights reserved

- 공백 처리 %s
  - → 선행 공백 무시
  - → 읽는 도중 공백에서 입력 종료
- 길이 제한 (중요)
  - 문자열 입력 시 반드시 제한 지정
  - 예: char name[20] → %19s ← 버퍼 오버플로 방지
- 반환값 검증
  - 반환값 = 성공적으로 읽은 항목 수 ← 입력 오류 검사 필수
  - Ex) if (scanf("%d", &value) != 1)
- 개행 문자 주의
  - scanf 후 '₩n'이 버퍼에 남을 수 있음
  - 특히 문자 입력에서 문제 발생 가능

16.3.3 형식 기반 입력 – scanf 계열

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

int main(void)
{
    char name[20];
    int value;

    // 1. 위험한 입력 (길이 제한 없음)
    // scanf("%s", name);  // 위험

    // 2. 안전한 문자열 입력

    int ret = scanf("%19s", name);
    if(ret != 1)    // 1개 입력 성공 여부 확인.
    {
        printf("문자열 입력 오류\n");
        return 1;
    }
    printf("이름: %s\n", name);
    // 3. 정수 입력 + 반환값 검증
    ret = scanf("%d", &value);
    if(ret != 1)   // 1개 입력 성공 여부 확인.
    {
        printf("정수 입력 오류\n");
        return 1;
    }
    printf("입력된 값: %d\n", value);
```

16.3.4 sscanf

Heesung Oh
Copyright All rights reserved

- `int sscanf (const* const buffer, const* const format, ...)`
  - 입력 대상 = 메모리 문자열
  - 문자열을 형식에 맞게 해석
  - scanf와 동일한 방식으로 동작
- 사용 예
  - 로그 파싱
  - 설정 파일 처리
  - 네트워크 데이터 해석

16.3.4 sscanf

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

int main(void)
{
    char data[] = "10 20";
    int x, y;

    if (sscanf(data, "%d %d", &x, &y) == 2)
    {
        printf("x=%d y=%d\n", x, y);
    }
    else
    {
        printf("파싱 실패\n");
    }

    return 0;
}
```

16.3.5 fscanf

Heesung Oh
Copyright All rights reserved

- `int fscanf (FILE *stream, const char* const format, ...)`
  - 파일 스트림에서 형식에 맞게 데이터 읽기
  - scanf → stdin 대상
  - fscanf → FILE 대상*
  - 반환값 = 성공적으로 읽은 항목 수

```
// 1. 파일 open
FILE* fp = fopen("data.txt", "r");
if (fp)
{
  int x, y;
  // 2. fscanf로 파일 읽기.
  if (fscanf(fp, "%d %d", &x, &y) == 2)
  {
    printf("x=%d y=%d\n", x, y);
  }
```

16.4 문자열 단위 입출력 함수 계열

Heesung Oh
Copyright All rights reserved

- **출력 계열**

함수	대상	설명
putchar	표준 출력 (stdout)	문자 1개 출력
fputc	파일 스트림	문자 1개 출력

- **입력 계열**

함수	대상	설명
getchar	표준 입력 (stdin)	문자 1개 입력
fgetc	파일 스트림	문자 1개 입력

16.4.1 문자열 단위 입출력 예제

Heesung Oh
Copyright All rights reserved

- getchar / putchar
  - 기본 스트림 사용 → stdin, stdout
- fgetc / fputc
  - FILE* 스트림 직접 지정

```c
// getchar()는 표준 입력(stdin)에서 문자 1개를 읽는다.
ch = getchar();

// EOF이면 루프 종료
if (EOF == ch)
    break;

// putchar()는 표준 출력(stdout)에 문자 1개를 출력한다.
putchar(ch);
```

```c
// stdin  : 표준 입력 스트림 (보통 키보드)
// stdout : 표준 출력 스트림 (보통 콘솔 화면)

// fgetc / fputc는 FILE*을 직접 지정하는 형태.
ch = fgetc(stdin);

// EOF이면 루프 종료
if (EOF == ch)
    break;

fputc(ch, stdout);
```

16.4.2 문자열 단위 입출력

Heesung Oh
Copyright All rights reserved

- **출력 계열**

함수	대상	설명
puts	표준 출력 (stdout)	문자열 출력 후 자동 개행
fputs	파일 스트림	문자열 출력 (개행 자동 추가 없음)

- **입력 계열**

함수	대상	설명
fgets	표준 입력 또는 파일	한 줄 입력 (개행 포함)
gets(X)	(C11 표준에서 제거)	버퍼 크기 확인 없음. 버퍼 오버플로 위험. fgets 사용
gets_s	표준 입력	일부 환경에서만 제공

16.4.2 문자열 단위 입출력 – 예제

Heesung Oh
Copyright All rights reserved

```c
#include <stdio.h>

int main(void)
{
    char buffer[100]={0};
    printf("문자열 입력: ");

    // fgets: 최대 n-1개의 문자를 읽고 마지막에 '\0'을 추가.
    fgets(buffer, sizeof(buffer), stdin);

    // 문자열 끝의 '\n' 제거
    buffer[strcspn(buffer, "\n")] = '\0';

    puts("입력한 내용:");
    puts(buffer);

    return 0;
}
```

16.5 Text File 입출력

Heesung Oh
Copyright All rights reserved

- Text File
  - 사람이 읽을 수 있는 문자 형태로 저장
  - fprintf,fputs 사용

16.5.1 fprintf

Heesung Oh
Copyright All rights reserved

- `int fprintf`
  `(FILE* const stream`
  `,const char* const format`
  `,...)`

  - 서식 문자열 사용 (%d, %s 등)
  - printf → 표준 출력 창에 출력
  - fprintf → 파일 스트림에 출력

```
FILE* fp = fopen("player.txt", "w");

fprintf(fp, "%s %d %d %d\n"
            , player[i].name
            , player[i].level
            , player[i].attack
            , player[i].defense);
```

16.5.2 fgets + sscanf

Heesung Oh
Copyright All rights reserved

```c
typedef struct Player {
    char name[20];
    int level;
    int attack;
    int defense;
} Player;

int main(void)
{
    FILE* fp = fopen("player.txt", "r");

    char buffer[512];
    Player p;

    while (1)
    {
        char* ptr = fgets(buffer, sizeof(buffer), fp);
        if (!ptr)
            break;
        // sscanf read count.

        int readCount = sscanf(buffer, "%19s %d %d %d"
                        , p.name, &p.level, &p.attack, &p.defense);
        if (readCount == 4)
        {
            printf("이름:%s 레벨:%d 공격력:%d 방어력:%d\n",
                  p.name, p.level, p.attack, p.defense);
        }
    }
```

16.6 바이너리 파일 입출력 (fread, fwrite)

Heesung Oh
Copyright All rights reserved

- 텍스트 파일
  - 사람이 읽기 쉬움
  - 문자열 변환 필요

- 바이너리 파일
  - 데이터 그대로 저장 (바이트 단위)
  - 변환 과정 없음

  - 처리 속도 빠름
  - 데이터 원형 그대로 유지
  - 구조체/배열을 메모리 형태 그대로 기록 가능

16.6.1~2 fwrite / fread

Heesung Oh
Copyright All rights reserved

- `size_t fwrite (const void* ptr, size_t size, size_t count, FILE* stream)`
  - 메모리 → 파일 기록
- `size_t fread (void* ptr, size_t size, size_t count, FILE* stream)`
  - 파일 → 메모리 읽기
- ptr : 데이터 시작 주소
  - size : 원소 크기 (바이트)
  - count : 원소 개수
  - stream: 파일 스트림
- 반환값: 실제로 처리된 원소 개수
- 오류 / EOF
  - 반환값 < count → 처리 실패
  - fread 반환값 = 0 → EOF 또는 오류 (feof / ferror로 구분)

16.6.3 copy binary file

Heesung Oh
Copyright All rights reserved

- 파일 복사
  - 특정 길이 버퍼 사용 4096

```
while (1) {
    bytes_read = fread(buffer, 1, BUFFER_SIZE, src);
    if (0 == bytes_read)
    {
        if (feof(src))
            printf("파일 끝 도달\n");
        else if (ferror(src))
            printf("파일 읽기 오류\n");

        break;
    }

    if (fwrite(buffer, 1, bytes_read, dest) != bytes_read)
    {
        printf("파일 쓰기 오류\n");
        break;
    }
}
```

16.7.1 파일 위치 표시자

Heesung Oh
Copyright All rights reserved

- `int fseek (FILE *stream, long offset, int origin)`
  - 파일은 기본적으로 순차 접근
  - 원하는 위치로 직접 이동 (랜덤 접근)

- 매개변수
  - offset : 이동할 바이트 수 → 양수: 앞으로 / 음수: 뒤로
  - origin : 기준 위치
    - SEEK_SET // 파일 시작
    - SEEK_CUR // 현재 위치
    - SEEK_END // 파일 끝

- 사용 예
  - `fseek(fp,  0, SEEK_SET);  // 처음`
  - `fseek(fp,  0, SEEK_END);  // 끝`
  - `fseek(fp, -10, SEEK_END);  // 끝에서 10바이트 앞`

16.7.2~3 현재 위치: ftell – 파일 크기 구하기

Heesung Oh
Copyright All rights reserved

- long ftell(FILE *stream)
  - 현재 파일 위치 반환
  - 파일 시작 기준 바이트 오프셋
  - 실패 시 -1
- 파일 크기 구하기
  - 현재 위치 저장
  - 파일 끝으로 이동
  - 위치 값 읽기 → 파일 크기
  - 원래 위치 복원

```
long GetFileSize(FILE* fp)
{
  if (!fp)
    return -1;

  // 현재 위치 저장
  long current = ftell(fp);
  if (current == -1)
    return -1;

  if (fseek(fp, 0, SEEK_END) != 0)
    return -1;

  // 파일 끝 위치 = 파일 크기
  long size = ftell(fp);

  if (fseek(fp, 0, SEEK_END) != 0)
    return -1;

  return size;
}
```

16.7.4 rewind

Heesung Oh
Copyright All rights reserved

- `void rewind(FILE* stream)`
  - 파일 위치를 처음으로 이동
  - fseek(fp, 0, SEEK_SET)과 동일하게 동작
  - EOF 및 오류 상태도 함께 초기화
  - 반환값 없음 → 오류 여부를 직접 확인할 수 없음

※ rewind = 파일 위치 처음으로 이동 + 상태 초기화

16.8 파일 입출력 응용

Heesung Oh
Copyright All rights reserved

- CSV 형식
  - 구조가 단순하고 사용이 쉬움
  - 쉼표(,)로 데이터를 구분
  - 텍스트 기반 데이터 파일 → 자료 구조 파악하기 용이
  - 게임 데이터 테이블 구성에 적합
  - 주의
    - 필드 내부에 쉼표(,)가 포함되면 추가 처리 필요

16.8.1~2 csv 스키마 – 구조체 자료 구조

Heesung Oh
Copyright All rights reserved

- 스키마

컬럼	타입	설명
id	key	아이템 고유 키
name	string	아이템 이름
description	string	아이템 설명
itemtype	string	아이템 분류(Weapon, Armor 등)
rarity	string	희귀도(Common, Rare 등)
price	int	가격
weight	float	무게

- 구조체 자료 구조

```
typedef struct ItemInfo {
    char id[16];
    char name[60];
    char description[120];
    char itemtype[32];
    char rarity[32];
    int price;
    float weight;
} ItemInfo;
```

16.8.3 CSV 파일 읽기 예제 (fgets + sscanf)

Heesung Oh
Copyright All rights reserved

```
// 아이템 정보 저장할 공간 확보.
ItemInfo* itemList = (ItemInfo*) malloc(sizeof(ItemInfo) * itemCount);
// 아이템 정보 초기화.
memset(itemList, 0, sizeof(ItemInfo) * itemCount);

// 아이템 정보(레코드) 읽기.
int itemIndex = 0;
while(1) {
    char* ptr = fgets(buffer, sizeof(buffer), fp);
    if(!ptr)
        break;

    if(itemIndex >= itemCount)
        break;
    // 아이템이 비어 있으면 빈곳이면 더이상 데이터가 없음을 인식하고
    if(4 >= strlen(buffer))
        break;

    // 아이템 정보 파싱.
    ItemInfo* pItem = &itemList[itemIndex];
    if(sscanf(buffer
        , "%15[^,],%59[^,],%119[^,],%31[^,],%31[^,],%d,%f"
        , pItem->id
        , pItem->name
        , pItem->description
        , pItem->itemtype
        , pItem->rarity
        , &pItem->price
        , &pItem->weight) == 7)
    {
        ++itemIndex;
```

16.8.4 Data 파일 Binary 예제

Heesung Oh
Copyright All rights reserved

- 바이너리 저장 / 로드
  - 텍스트 방식 대비 빠르고 단순
  - 숫자 ↔ 문자열 변환 없음
  - 구조체 메모리를 그대로 저장 → 처리 속도 향상, 코드 단순화
- 저장 구조
  - 플레이어 수 저장
  - 플레이어 배열 전체 저장
- 로드 구조
  - 저장 구조와 1:1 대응
  - 플레이어 수 읽기 → 메모리 동적 할당
  - 배열 전체 읽기

16.8.4.1 바이너리 저장 함수

Heesung Oh
Copyright All rights reserved

```
int SavePlayerData(const char* fileName, const Player* player, int count)
{

    // 1. 저장 파일 열기.
    fp = fopen(fileName, "wb");
    if(!fp) {
        printf("저장 파일 %s 열기 실패\n", fileName);
        goto ERR_HANDLE;
    }
    // 2. 플레이어 숫자 저장.
    if(1 != fwrite(&count, sizeof(int), 1, fp)) {
        goto ERR_HANDLE;
    }
    // 3. 구조체 리스트 전체 저장.
    if(count != (int)fwrite(player, sizeof(Player), count, fp)) {
        goto ERR_HANDLE;
    }
    // 파일 닫기.
    fclose(fp);
    return count;
ERR_HANDLE:
```

16.8.4.2 바이너리 로드 함수

Heesung Oh
Copyright All rights reserved

```
int LoadPlayerData(const char* fileName, Player** player) {
    // 1. 저장 파일 열기.
    FILE* fp = fopen(fileName, "rb");
    if(!fp) {
        printf("파일: %s 열기 실패\n", fileName);
        goto ERR_HANDLE;
    }
    // 2. 플레이어 숫자 읽기.
    if(1 != fread(&player_count, sizeof(int), 1, fp)) {
        printf("파일: %s에서 플레이어 수 읽기 실패\n", fileName);
        goto ERR_HANDLE;
    }
    if(0>=player_count) {
        printf("파일: %s에서 읽을 플레이어 데이터가 없습니다.\n", fileName);
        goto ERR_HANDLE;
    }
    // 3. 파일에서 읽어온 플레이어 데이터를 임시로 저장할 버퍼 동적 할당.
    tmpPlayer = malloc(sizeof(Player) * player_count);
    if (!tmpPlayer) {
        printf("메모리 생성 실패: %s\n", fileName);
        goto ERR_HANDLE;
    }
    // 4. 플레이어 데이터 통째로 로드.
    readCount = fread(tmpPlayer, sizeof(Player), player_count, fp);
    // 5. 읽어온 데이터 개수가 원소 개수와 일치하는지 확인.
    if (readCount != (size_t)player_count) {
        printf("파일: %s에서 플레이어 데이터를 읽는 중 오류 발생\n", fileName);
        goto ERR_HANDLE;
    }
    //6. 읽어온 플레이어 데이터 버퍼 주소 전달.
    *player = tmpPlayer;
    fclose(fp);
    return player_count;
ERR_HANDLE:
```

16.8.4.3 저장/로드 실행

Heesung Oh
Copyright All rights reserved

```
int main(void)
{
    Player player_src[] =
    {
        {"Knight", 10, 25, 15},
        {"Archer", 8, 18, 10},
        {"Wizard", 12, 30, 5},
        {"Paladin", 15, 28, 22}
    };

    // 원본 플레이어 데이터를 바이너리 파일로 저장.
    int player_count = sizeof(player_src)/sizeof(player_src[0]);
    SavePlayerData("player.dat", player_src, player_count);

    // 파일에 저장된 플레이어 데이터 읽기.
    Player* dest_buf = NULL;
    int dest_count = LoadPlayerData("player.dat", &dest_buf);

    // 읽어온 플레이어 데이터 출력.
    printf("이름\t\t\t레벨\t공격력\t방어력\n");
    for(int i=0; i<dest_count; ++i)
    {
        Player* ptr = &dest_buf[i];
        printf("%19s\t%4d\t%4d\t%4d\n"
            , ptr->name, ptr->level, ptr->attack, ptr->defense);
    }
```

학습 정리

Heesung Oh
Copyright All rights reserved

- **스트림(Stream)**
  - 연속된 바이트 흐름
  - 콘솔 / 파일 / 메모리 → 동일한 방식으로 처리
- **파일과 자원 관리**
  - 파일 = OS 자원: fopen → 사용 → fclose
  - 메모리와 동일한 흐름: malloc → 사용 → free
- **텍스트 vs 바이너리**
  - 텍스트: 사람이 읽기 가능, 변환 필요
  - 바이너리: 바이트 그대로 저장 → 속도 빠름 / 구조 단순
- **파일 처리**
  - EOF vs 오류 구분 → feof, ferror
  - 반환값 검사 필수

Heesung Oh
Copyright All rights reserved

표준 출력

함수	대상	의미
printf	stdout	형식 문자열에 맞춰 표준 출력으로 출력
fprintf	FILE*	지정한 파일 스트림에 형식에 맞춰 출력
sprintf	char* 버퍼	형식에 맞춰 문자열 버퍼에 저장
snprintf	char* 버퍼	버퍼 크기를 제한하여 안전하게 문자열 저장
putchar	stdout	문자 1개 출력
fputc	FILE*	지정한 파일 스트림에 문자 1개 출력
puts	stdout	문자열 출력 후 자동 개행
fputs	FILE*	문자열 출력 (자동 개행 없음)

Heesung Oh
Copyright All rights reserved

표준 입력

함수	대상	의미
scanf	stdin	형식에 맞게 표준 입력에서 읽기
fscanf	FILE*	지정한 파일 스트림에서 형식에 맞게 읽기
sscanf	char* 문자열	메모리 문자열을 형식에 맞게 해석(파싱)
getchar	stdin	문자 1개 입력
fgetc	FILE*	지정한 파일 스트림에서 문자 1개 입력
fgets	stdin / FILE*	한 줄 문자열 입력 (버퍼 크기 제한)

Heesung Oh
Copyright All rights reserved

- **파일 관리 및 바이너리 입출력**

함수	대상	의미
fopen	파일	파일 열기 (스트림 생성)
fclose	파일	파일 닫기 (자원 반환)
fread	FILE*	파일에서 바이트 단위로 읽기
fwrite	FILE*	파일에 바이트 단위로 쓰기
fseek	FILE*	파일 위치 표시자 이동
ftell	FILE*	현재 파일 위치 반환
feof	FILE*	파일 끝 도달 여부 확인
ferror	FILE*	파일 오류 발생 여부 확인
fflush	FILE*	해당 스트림의 출력 버퍼를 즉시 비움

실습 과제

Heesung Oh
Copyright All rights reserved

- 로그 파일 생성 및 분석
- CSV 읽기 및 정렬 + 바이너리 저장/로드
- 파일 크기 계산 및 파일 복사
- 구조체 랜덤 접근 수정